BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

KHOA ĐIỆN – ĐIỆN TỬ

---------- ✦ ----------

BÀI TẬP LAB 3

Môn: Hệ thống nhúng

ĐỀ SỐ: 5

ĐỀ BÀI: THIẾT KẾ MẠCH PHÁT HIỆN TÉ NGÃ SỬ DỤNG ESP32, MPU6050, FREERTOS VÀ MQTT

Giảng viên hướng dẫn:	Ths. Võ Quang Sơn
Sinh viên thực hiện:	Nguyễn Ngọc Thọ
Mã sinh viên:	231432360
Lớp:	Kỹ thuật Điện tử-Tin học công nghiệp 2 K64

Hà Nội 2026

MỤC LỤC

LỜI MỞ ĐẦU 3
I. PHẦN MỞ ĐẦU 4
II. CƠ SỞ LÝ THUYẾT VỀ RTOS VÀ FREERTOS 5
III. CÁC CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK 5
IV. CẢM BIẾN MPU6050 6
V. GIAO THỨC MQTT VÀ MQTT BROKER 6
VI. CÁC BƯỚC THỰC HIỆN BÀI LAB 7
6.1. Kiến trúc hệ thống 7
6.2. Sơ đồ mạch trong Wokwi 7
6.3. Triển khai FreeRTOS trên ESP32 8
6.4. Thuật toán phát hiện và điều khiển cảnh báo 9
6.5. Giao diện Web FallGuard 10
VII. KẾT QUẢ ĐẠT ĐƯỢC 11
7.1. Kết quả mô phỏng trên Wokwi 11
7.2. Kết quả trên giao diện Web 12
7.3. Bảng kiểm tra chức năng 13
KẾT LUẬN 14
TÀI LIỆU THAM KHẢO 15

LỜI MỞ ĐẦU

Một cú té ngã có thể làm cơ thể thay đổi tốc độ và tư thế trong thời gian ngắn. Khi cảm biến chuyển động được gắn cố định, những thay đổi đó có thể quan sát qua gia tốc ba trục và góc nghiêng ước lượng. Tuy nhiên, các hoạt động thường ngày như ngồi xuống nhanh hoặc cúi người cũng có thể tạo ra biến động rõ rệt. Để giảm nhầm lẫn trong bài thực hành, hệ thống theo dõi cả sự thay đổi gia tốc ban đầu và tư thế duy trì sau biến động.

Từ yêu cầu của Đề 5 môn Hệ thống nhúng, em xây dựng FallGuard trên Wokwi với ESP32, cảm biến MPU6050, đèn LED và còi. ESP32 đọc gia tốc qua I²C, tính độ lớn gia tốc và góc nghiêng, rồi hiển thị số liệu trên Serial Monitor. Bộ phát hiện ghi nhận biến động khi gia tốc qua ngưỡng và tiếp tục kiểm tra trong một khoảng thời gian. Điều kiện xác nhận kết hợp độ nghiêng, độ chênh góc so với trước biến động và mức gia tốc ổn định.

Chương trình sử dụng FreeRTOS để chia công việc giữa hai lõi của ESP32. Core 1 đọc cảm biến và xử lý cảnh báo. Core 0 đảm nhận kết nối Wi-Fi và truyền dữ liệu MQTT. Mỗi mẫu được chuyển tới hai hàng đợi riêng để tác vụ truyền và tác vụ cảnh báo có thể xử lý theo nhịp của mình. Cách tổ chức này giúp phần đọc cảm biến và điều khiển cảnh báo tại chỗ tiếp tục chạy trong lúc tác vụ mạng thử kết nối lại.

Giao diện HTML nhận dữ liệu qua MQTT để hiển thị gia tốc, góc nghiêng, trạng thái kết nối và hai biểu đồ theo thời gian. Khi có cảnh báo ở chế độ AUTO, ESP32 bật LED và còi, còn giao diện hiển thị dải cảnh báo và phát âm thanh. Chế độ MANUAL cho phép điều khiển riêng từng ngõ ra. Dashboard cập nhật trạng thái sau khi ESP32 gửi bản tin phản hồi, giúp người thử kiểm tra lệnh đã được thực hiện.

Báo cáo trình bày sơ đồ đấu nối, cách tổ chức các tác vụ, điều kiện phát hiện té ngã và quy trình thử trên Wokwi. Các kết quả gồm mẫu MPU6050 khi đứng yên, kịch bản té ngã dùng dữ liệu tổng hợp và thử điều khiển thủ công. Phạm vi đánh giá là mạch và phần mềm mô phỏng. Kết quả phản ánh hoạt động của nguyên mẫu trong các kịch bản đã thử. Độ nhạy và tỷ lệ báo nhầm cần được đánh giá thêm trên thiết bị vật lý.

I. PHẦN MỞ ĐẦU

1.1. Đặt vấn đề

Một đỉnh gia tốc có thể xuất hiện khi đi nhanh, thay đổi tư thế hoặc va chạm nhẹ. Do đó, bài thực hành kết hợp dấu hiệu gia tốc vượt ngưỡng với góc nghiêng ổn định sau biến động. Hệ thống cũng cần theo dõi tín hiệu cảm biến liên tục khi Wi-Fi hoặc MQTT đang thử kết nối lại.

1.2. Mục tiêu và phạm vi

- Lắp mạch ESP32, MPU6050, LED và còi trên Wokwi. Hiển thị gia tốc ba trục, độ lớn gia tốc và góc nghiêng qua Serial.
- Chạy tác vụ truyền dữ liệu trên Core 0. Chạy tác vụ đọc cảm biến và tác vụ cảnh báo trên Core 1. Dùng hai hàng đợi để chuyển mẫu đến đúng tác vụ.
- Dùng Wi-Fi và MQTT để hiển thị dữ liệu dạng số, hai biểu đồ và trạng thái thiết bị trên HTML. Hai nút riêng điều khiển LED và còi khi chọn MANUAL.
- Đánh giá ở trạng thái đứng yên, kịch bản đi lại, kịch bản té ngã tổng hợp và chế độ điều khiển thủ công. Phạm vi kiểm thử là phần mềm và mạch mô phỏng.

II. CƠ SỞ LÝ THUYẾT VỀ RTOS VÀ FREERTOS

2.1. Hệ điều hành thời gian thực

RTOS lập lịch tác vụ theo độ ưu tiên và trạng thái sẵn sàng. Một tác vụ có thể nhường CPU khi chờ đến chu kỳ tiếp theo hoặc chờ dữ liệu trong hàng đợi. Trong bài Lab, thời gian lấy mẫu mục tiêu là 50 ms và thời gian phát bản tin mục tiêu là 400 ms khi đã kết nối. Đây là thông số thiết kế của firmware.

2.2. FreeRTOS trên ESP32

ESP32 trong dự án có hai lõi. Hàm xTaskCreatePinnedToCore() tạo tác vụ gắn với một lõi cụ thể [2]. Phần kết nối Wi-Fi và MQTT do Core 0 thực hiện. Core 1 lấy mẫu và xử lý ngõ ra, với độ ưu tiên của tác vụ cảnh báo cao hơn tác vụ đọc cảm biến.

III. CÁC CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK

3.1. Hàng đợi dữ liệu

FreeRTOS sao chép phần tử vào queue. Hai queue chính giữ một mẫu mới nhất bằng xQueueOverwrite(). Mỗi tác vụ tiêu thụ nhận bản sao riêng, vì thao tác nhận từ queue lấy phần tử ra khỏi queue đó. Nếu mạng chậm, các mẫu mới ghi đè mẫu chưa gửi.

Bảng 3.1. Hàng đợi và cơ chế đồng bộ trong chương trình

Cơ chế	Dung lượng	Vai trò
dataQueue	1 Sample	TaskDocData → TaskTruyenData. Cung cấp mẫu mới nhất cho MQTT.
dataAlert	1 Sample	TaskDocData → TaskCanhBao. Cung cấp mẫu cho bộ phát hiện.
commandQueue	16 Command	Chuyển lệnh đổi chế độ, điều khiển LED/còi và ACK.
simulationQueue	1 bool	Chuyển yêu cầu START/STOP đến TaskDocData.
stateMutex	Mutex	Bảo vệ trạng thái dùng chung DeviceState khi đọc và ghi.

3.2. Trạng thái dùng chung

TaskCanhBao cập nhật trạng thái thiết bị và bộ đếm sự kiện. TaskTruyenData lấy bản sao trạng thái khi đóng gói JSON. Mutex giới hạn truy cập đồng thời vào vùng dữ liệu này. Mã gửi MQTT chỉ chạy trong TaskTruyenData.

IV. CẢM BIẾN MPU6050

MPU6050 tích hợp gia tốc kế và con quay hồi chuyển ba trục, giao tiếp với ESP32 qua I²C [3]. Firmware đọc dữ liệu gia tốc bằng thư viện Adafruit MPU6050, chia kết quả m/s² cho 9,80665 để biểu diễn theo g. Cảm biến được cấu hình dải đo gia tốc ±16 g và bộ lọc 21 Hz.

$$A = \sqrt{(a_x^2 + a_y^2 + a_z^2)} \quad \bullet \quad \theta = atan2(\sqrt{(a_x^2 + a_y^2)}, a_z) \times 180/\pi$$

Độ lớn A tính theo g. Góc θ nằm trong khoảng 0–180° và được ước lượng từ hướng của vector gia tốc so với trục +Z. Khi cảm biến nằm yên, gia tốc theo ba trục lần lượt là 0 g, 0 g và 1 g, nên A = 1 g và θ = 0°. Gia tốc do chuyển động có thể làm góc ước lượng dao động, vì vậy thuật toán chỉ xác nhận khi mẫu đã ổn định.

V. GIAO THỨC MQTT VÀ MQTT BROKER

MQTT truyền bản tin qua broker theo mô hình publish/subscribe. ESP32 kết nối Wokwi-GUEST, sau đó dùng MQTT/TCP tại broker.emqx.io:1883. Trình duyệt dùng MQTT qua WSS tại wss://broker.emqx.io:8084/mqtt [4], [5]. Cả hai dùng topic gốc fd-demo-81472.

Bảng 5.1. Topic MQTT và giá trị bản tin

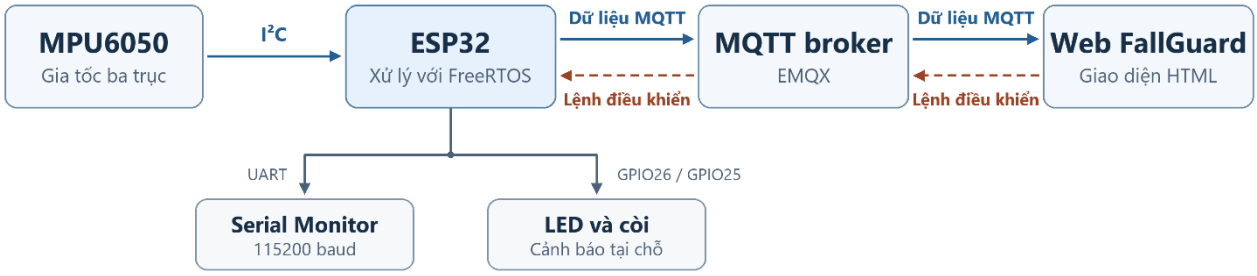
Hậu tố topic	Bên gửi	Giá trị
/telemetry	ESP32	JSON chứa phép đo, chế độ, trạng thái và bộ đếm.
/cmd/mode	Web	AUTO hoặc MANUAL
/cmd/led	Web	ON hoặc OFF
/cmd/buzzer	Web	ON hoặc OFF
/cmd/simulate	Web	START hoặc STOP
/cmd/reset	Web	ACK để kết thúc cảnh báo hiện hành

Các lệnh gửi với QoS 0 và không lưu retain. Web chờ telemetry để xác nhận trạng thái ESP32 sau mỗi lệnh. Các trường ax, ay, az, accel, tilt mang dữ liệu đo. Các trường fall, led, buzzer, mode, events, simEvents và simulated cho biết trạng thái và nguồn dữ liệu.

VI. CÁC BƯỚC THỰC HIỆN BÀI LAB

6.1. Kiến trúc hệ thống

ESP32 đọc MPU6050 qua I²C và xuất số liệu ra Serial. Tác vụ cảnh báo điều khiển hai GPIO. Qua MQTT, ESP32 gửi dữ liệu lên broker và nhận lệnh điều khiển từ dashboard.



Hình 6.1. Kiến trúc tổng quát hệ thống FallGuard

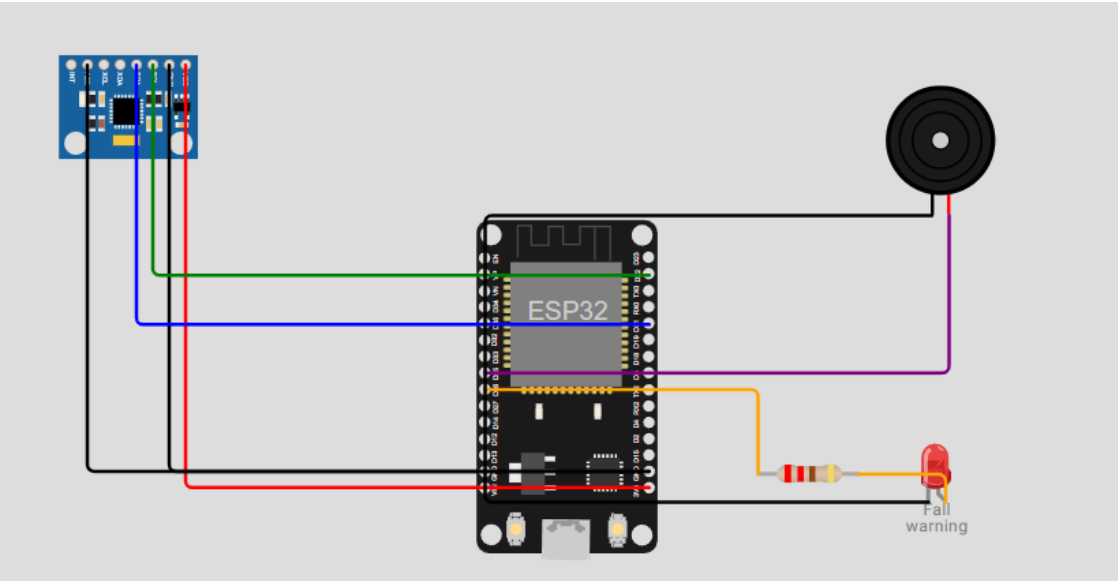
6.2. Sơ đồ mạch trong Wokwi

Tạo mạch với ESP32 DevKit v1, MPU6050, LED đỏ, điện trở 220 Ω và còi piezo. Nối chung GND. Kéo AD0 xuống GND để dùng địa chỉ I²C 0x68.

Bảng 6.1. Kết nối các linh kiện trong Wokwi

Linh kiện	Chân hoặc đường tín hiệu	Kết nối trên ESP32
MPU6050	VCC, GND, AD0	3V3, GND, GND
MPU6050	SDA, SCL	GPIO21, GPIO22
LED đỏ	Anode qua 220 Ω, cathode	GPIO26, GND
Còi piezo	Chân 2, chân 1	GPIO25, GND

Mở dự án Wokwi [6] để xem sơ đồ theo diagram.json. Serial Monitor dùng tốc độ 115200 baud.



Hình 6.2. Mạch ESP32, MPU6050, LED và còi trên Wokwi

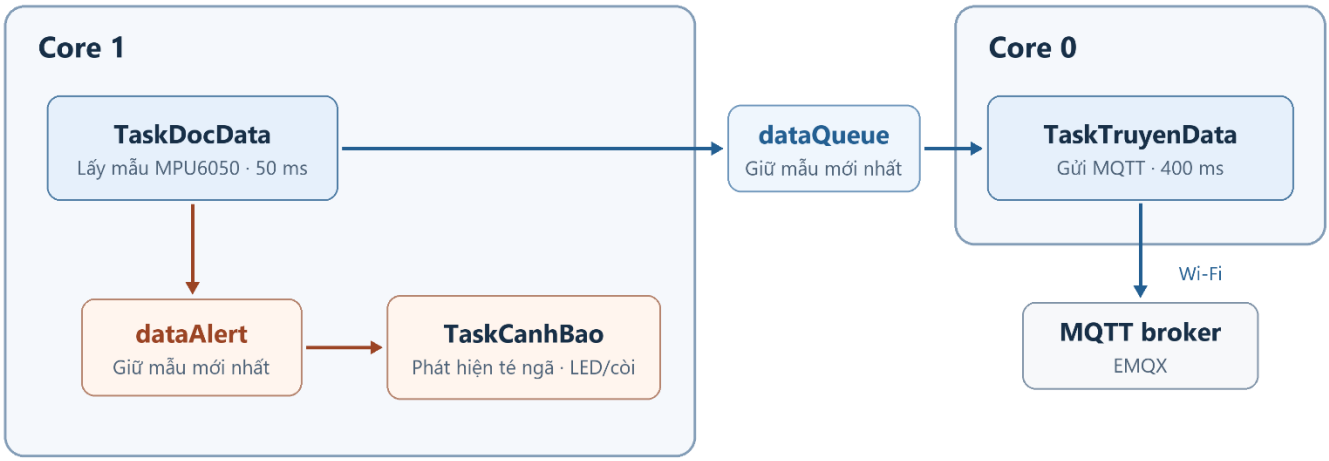
6.3. Triển khai FreeRTOS trên ESP32

Trong setup(), chương trình cấu hình Serial, GPIO và I²C, khởi tạo MPU6050, sau đó cấp phát queue và mutex trước khi tạo tác vụ. loop() nhường CPU bằng vTaskDelay().

Bảng 6.2. Phân công tác vụ FreeRTOS

Tác vụ	Lõi	Ưu tiên	Hoạt động
TaskDocData	1	2	Đọc và tính toán mỗi 50 ms. Ghi mẫu vào dataQueue và dataAlert. In Serial theo chu kỳ 400 ms.
TaskCanhBao	1	3	Đọc lệnh, chờ dataAlert tối đa 20 ms mỗi vòng, chạy bộ phát hiện và điều khiển GPIO.
TaskTruyenData	0	1	Quản lý Wi-Fi/MQTT. Khi kết nối thành công, lấy mẫu từ dataQueue và gửi JSON theo chu kỳ mục tiêu 400 ms.

Trong TaskDocData, vTaskDelayUntil() giữ mốc lấy mẫu. Khi mạng mất kết nối, TaskTruyenData thử lại Wi-Fi hoặc MQTT. Hai tác vụ trên Core 1 vẫn có thể đọc cảm biến và xử lý cảnh báo tại chỗ.



Hình 6.3. Trao đổi dữ liệu giữa ba tác vụ FreeRTOS

Trích mã khởi tạo ba tác vụ trong sketch.ino

```
configASSERT(xTaskCreatePinnedToCore(
    TaskCanhBao, "TaskCanhBao", 4096, nullptr, 3, nullptr, 1) == pdPASS);
configASSERT(xTaskCreatePinnedToCore(
    TaskDocData, "TaskDocData", 4096, nullptr, 2, nullptr, 1) == pdPASS);
configASSERT(xTaskCreatePinnedToCore(
    TaskTruyenData, "TaskTruyenData", 6144, nullptr, 1, nullptr, 0) == pdPASS);
```

Đối số cuối của xTaskCreatePinnedToCore() là lõi chạy tác vụ. Giá trị 4096 và 6144 là dung lượng stack tính theo byte trong ESP-IDF [2].

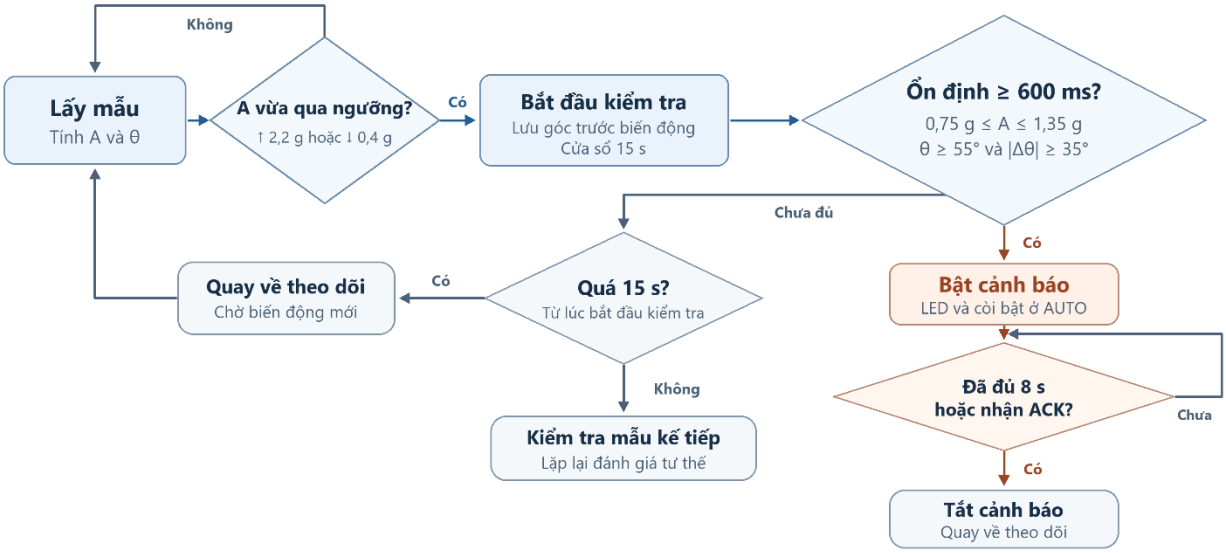
6.4. Thuật toán phát hiện và điều khiển cảnh báo

TaskCanhBao gọi FallDetector trong detector.h với độ lớn gia tốc A, góc θ, thời điểm lấy mẫu và nguồn dữ liệu. Khi bắt đầu hoặc đổi nguồn, bộ phát hiện lưu mẫu đầu và chờ 800 ms trước khi tìm biến động.

Bảng 6.3. Điều kiện phát hiện té ngã

Giai đoạn	Điều kiện xử lý
Phát hiện biến động	A vượt lên trên 2,2 g hoặc hạ xuống dưới 0,4 g so với mẫu trước. Lưu góc của mẫu trước biến động.
Xác nhận tư thế	Trong cửa sổ khoảng 15 s, θ ≥ 55°, độ chênh góc so với trước biến động ≥ 35° và 0,75 g ≤ A ≤ 1,35 g liên tục trong ít nhất 600 ms.
Kích hoạt cảnh báo	Bật cờ fall, tăng bộ đếm theo nguồn mẫu. Cảnh báo tự hết sau khoảng 8 s khi tiếp tục lấy mẫu, hoặc kết thúc khi nhận ACK.

Ở AUTO, GPIO26 bật LED và GPIO25 phát còi 1 kHz khi cờ cảnh báo được kích hoạt. Ở MANUAL, hai nút gửi lệnh độc lập cho LED và còi. Khi chuyển sang MANUAL, cả hai ngõ ra thủ công bắt đầu ở trạng thái tắt.



Hình 6.4. Lưu đồ phát hiện và xác nhận té ngã

Kịch bản kiểm thử trên ESP32

Khi nhận START ở AUTO, TaskDocData thay mẫu cảm biến bằng vector tổng hợp theo trình tự đứng yên → A = 0,10 g → A = 2,80 g → nằm nghiêng với A ≈ 1,04 g, θ ≈ 73,3°. Chuỗi này đi qua cùng hai queue và thuật toán phát hiện. Khi nhận STOP hoặc hết kịch bản, chương trình trở về đọc MPU6050.

6.5. Giao diện Web FallGuard

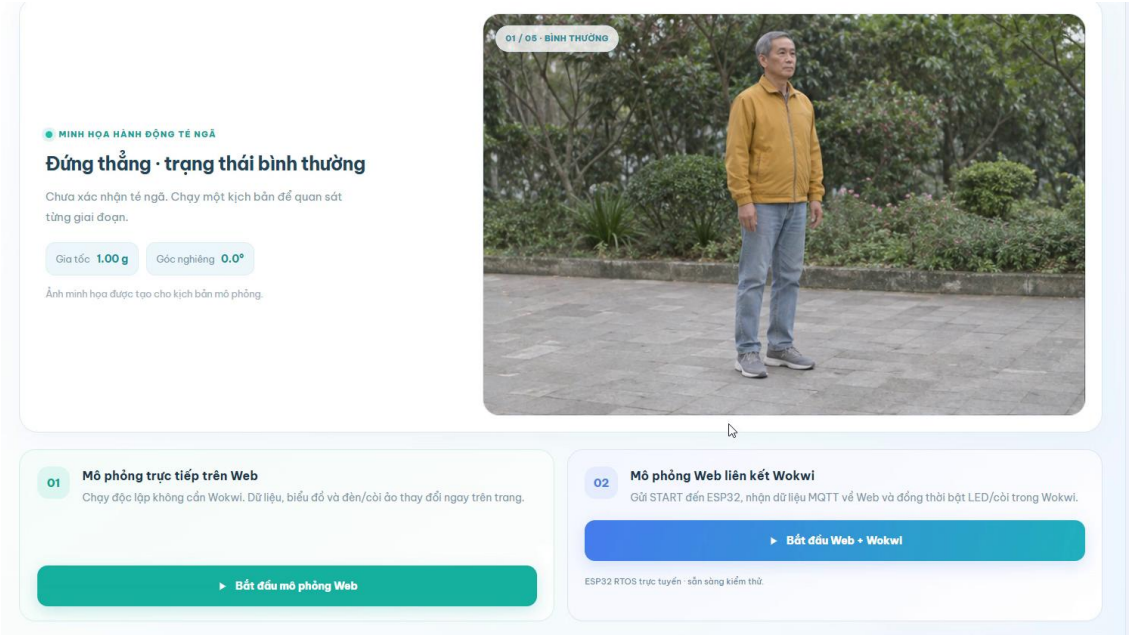
Dashboard FallGuard được đóng gói dưới dạng HTML. Có thể mở dashboard.html hoặc index.html bằng trình duyệt. Giao diện hiển thị Ax, Ay, Az, A và θ dạng số, cùng biểu đồ gia tốc và biểu đồ góc nghiêng theo thời gian. Ảnh minh họa thay đổi theo giai đoạn đứng, mất thăng bằng, chạm đất và nằm nghiêng.

Các ô giá trị dùng lớp CSS metric. JavaScript cập nhật thẻ strong có ID ax, ay, az, accel và tilt từ telemetry. Hai thẻ button có ID ledToggle và buzzerToggle nhận thao tác bật tắt thiết bị.

Bảng 6.4. Hai nguồn dữ liệu trên dashboard

Nguồn	Dữ liệu	Điều khiển thiết bị
Mô phỏng Web	Trình duyệt tạo mẫu cho kịch bản té ngã hoặc đi lại.	Các nút LED và còi thiết bị bị khóa.
Web liên kết Wokwi	Nhận telemetry từ ESP32 qua MQTT. Bài thử START dùng mẫu tổng hợp trong firmware.	Hai nút LED và còi hoạt động khi ESP32 trực tuyến và đang ở MANUAL.

Khi có cảnh báo, dashboard hiện dải đỏ và phát chuỗi còi ba nhịp tự động. Âm thanh trình duyệt phục vụ người theo dõi. Nút điều khiển còi ở MANUAL gửi lệnh tới còi nối với ESP32.



Hình 6.5. FallGuard nhận dữ liệu Wokwi khi MPU6050 đứng yên

Trình tự chạy liên kết Wokwi

1. Chạy mô phỏng Wokwi và chờ Serial báo MQTT đã kết nối.
2. Mở dashboard, chọn Web liên kết Wokwi và đợi telemetry đầu tiên.
3. Chọn MANUAL để thử từng nút. Quay lại AUTO rồi chạy kịch bản té ngã.

VII. KẾT QUẢ ĐẠT ĐƯỢC

7.1. Kết quả mô phỏng trên Wokwi

Trong phiên kiểm thử ngày 09/11/2026, Serial ghi TaskDocData và TaskCanhBao chạy Core 1, TaskTruyenData chạy Core 0. Khi MPU6050 đứng yên, Ax = 0 g, Ay = 0 g, Az = 1 g, A = 1,00 g, góc nghiêng = 0,0°. Nhật ký cũng ghi MQTT đã kết nối và đăng ký các topic lệnh [7].

```
[RTOS] TaskCanhBao core=1
[RTOS] TaskDocData core=1 period=50ms
[RTOS] TaskTruyenData core=0
[READ C1] #1 X=+0.00g Y=+0.00g Z=+1.00g | A=1.00g Tilt=0.0 deg | MPU6050
[READ C1] #9 X=+0.00g Y=+0.00g Z=+1.00g | A=1.00g Tilt=0.0 deg | MPU6050
[READ C1] #17 X=+0.00g Y=+0.00g Z=+1.00g | A=1.00g Tilt=0.0 deg | MPU6050
[READ C1] #25 X=+0.00g Y=+0.00g Z=+1.00g | A=1.00g Tilt=0.0 deg | MPU6050
```

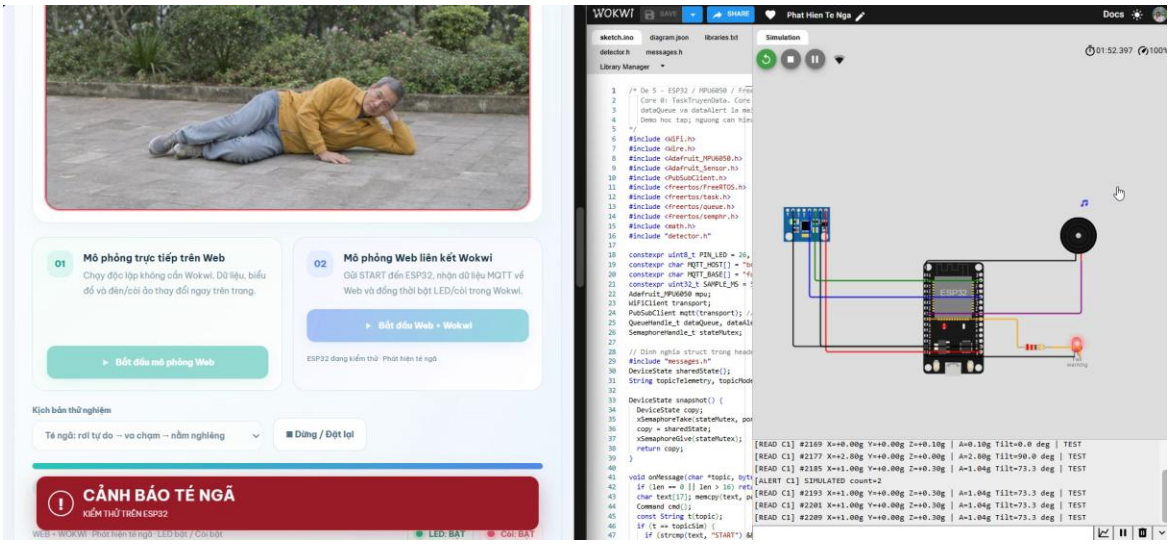
Hình 7.1. Serial ghi ba tác vụ FreeRTOS và mẫu MPU6050 khi đứng yên (video demo, 03:10)

7.1.1. Ảnh hưởng khi IoT Gateway mất kết nối

Ở lần chạy có lỗi "Connection to IoT Gateway … failed", Serial vẫn in mẫu MPU6050. Dashboard hiển thị "Đang chờ ESP32" vì chưa nhận telemetry. Có thể đối chiếu nhật ký Serial để phân biệt bước đọc cảm biến với bước kết nối và truyền dữ liệu qua gateway.

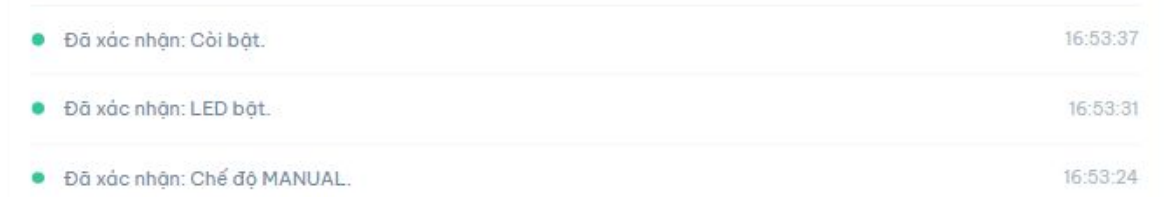
7.2. Kết quả trên giao diện Web

Khi liên kết Wokwi hoạt động, dashboard cập nhật hai biểu đồ và trạng thái ngõ ra theo telemetry. Trong một phiên kiểm thử, cảnh báo được xác nhận ở A khoảng 1,04 g và θ khoảng 73,3° sau pha va chạm. Cờ fall bật khiến dashboard chuyển sang trạng thái ALERT và hiển thị dải cảnh báo đỏ. Ở AUTO, LED và còi ESP32 cùng bật [7].



Hình 7.2. Cảnh báo trên FallGuard cùng LED và còi đang bật trên Wokwi (video demo, 05:20)

Ở MANUAL, dashboard gửi lệnh riêng cho từng ngõ ra và cập nhật công tắc sau khi ESP32 phản hồi. Có thể bật LED trong khi còi tắt, rồi đổi trạng thái từng nút để kiểm tra tính độc lập.



Hình 7.3. Nhật ký FallGuard xác nhận chế độ MANUAL và hai lệnh bật LED, bật còi



Hình 7.4. Hai biểu đồ gia tốc và góc nghiêng từ kịch bản kiểm thử trên ESP32

7.3. Bảng kiểm tra chức năng

Các kết quả dưới đây tổng hợp từ nhật ký kiểm thử firmware và giao diện [7], [8]. Các ca dùng mẫu tổng hợp được ghi rõ nguồn để phân biệt với phép đọc MPU6050.

Bảng 7.1. Kịch bản và kết quả kiểm thử

STT	Kịch bản	Kết quả quan sát
1	Khởi động Wokwi	Ba tác vụ xuất hiện trên Serial với sơ đồ lõi 1–1–0. Mạch mô phỏng khởi động thành công.
2	MPU6050 đứng yên	Serial và web hiển thị A ≈ 1,00 g và θ ≈ 0°. Không kích hoạt báo động.
3	Web nhận MQTT	Khi broker và gateway hoạt động, dashboard nhận telemetry và hai biểu đồ cập nhật.
4	Đi lại trong mô phỏng Web	Chuỗi mẫu đi lại hiển thị trên đồ thị. Bộ phát hiện không kích hoạt cảnh báo.
5	Té ngã tổng hợp trên ESP32	Trong phiên kiểm thử hai lần, Serial báo SIMULATED count=1 và count=2. simEvents = 2, events = 0. LED/còi bật ở AUTO.
6	Điều khiển MANUAL	Chuyển chế độ, bật LED và bật còi bằng hai lệnh riêng. Web nhận trạng thái xác nhận từ telemetry.
7	Xác nhận cảnh báo	ACK được ESP32 xử lý. Telemetry báo hết cảnh báo, bộ đếm sự kiện được giữ.

7.4. Đánh giá

Phép thử xác nhận đường dữ liệu từ ESP32 đến dashboard và đường lệnh từ web về mạch khi MQTT trực tuyến. Hai sự kiện trong bộ đếm simEvents do firmware tạo mẫu, nên dùng để đánh giá việc truyền dữ liệu và phản ứng của bộ phát hiện trong mô phỏng.

Sai số góc khi chuyển động phụ thuộc hướng lắp cảm biến và gia tốc động. Hai queue một phần tử ưu tiên dữ liệu mới, nên dashboard có thể bỏ qua một xung ngắn giữa hai lần gửi. Kết quả mô phỏng chưa đủ để xác định độ nhạy và tỷ lệ báo nhầm trên thiết bị thật.

KẾT LUẬN

Bài Lab đã xây dựng mạch phát hiện té ngã trên Wokwi với ESP32, MPU6050, LED và còi. Chương trình FreeRTOS chia việc lấy mẫu, truyền dữ liệu và xử lý cảnh báo theo đúng hai lõi được yêu cầu. Hàng đợi dataQueue cấp mẫu cho đường truyền MQTT, còn dataAlert cấp mẫu cho bộ phát hiện.

Dashboard HTML hiển thị gia tốc và góc nghiêng bằng số và biểu đồ, hỗ trợ điều khiển thủ công hai ngõ ra, đồng thời thể hiện cảnh báo bằng hình, màu và âm thanh. Kịch bản tổng hợp đã kiểm tra được việc phát hiện và điều khiển ngõ ra. Bước đánh giá tiếp theo là thử trên mạch vật lý, hiệu chỉnh vị trí lắp và thu mẫu nhiều hoạt động thường ngày để đo tỷ lệ báo nhầm.

TÀI LIỆU THAM KHẢO

[1] Đề 5, Bài tập Lab 3 môn Hệ thống nhúng. Yêu cầu thiết kế mạch phát hiện té ngã.

[2] Espressif Systems. ESP-IDF Programming Guide. FreeRTOS (IDF) – ESP32

[3] Wokwi Docs. wokwi-mpu6050 – 6-Axis Accel & Gyro Sensor

[4] Wokwi Docs. ESP32 Wi-Fi Networking

[5] EMQX. Public MQTT Broker – thông số TCP và WSS

[6] Dự án mô phỏng FallGuard. ESP32, MPU6050 và FreeRTOS trên Wokwi

[7] FallGuard-RTOS. Mã sketch.ino, detector.h, messages.h, diagram.json và nhật ký KIEM-THU.md ngày 09/11/2026.

[8] FallGuard. Mã nguồn, giao diện Web Dashboard và video mô phỏng. Kho dự án trên GitHub